Dokumentasi Praktikum Pemrograman Mobile

Pertemuan 6



Nama:

Muhammad Yusuf Bagus Syafaat

NIM:
20240801041

UNIVERSITAS ESA UNGGUL TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

2026

1. Pendahuluan

Praktikum Pertemuan 6 membahas pengelolaan tema aplikasi, navigasi ant halaman, dan animasi pada Flutter. Implementasi utama menggabungkan tema terang/gelap dan pilihan warna, named routes dengan pengiriman argumen, transisi Hero, animasi implisit dan eksplisit, serta navigasi tab bawah. Lima berkas latihan mandiri memperluas aplikasi dengan penyimpanan preferensi, Drawer, transisi halaman khusus, penanganan rute tidak ditemukan, dan animasi opacity.

2. Tujuan Praktikum

- Membuat tema terang dan gelap serta pilihan warna menggunakan ThemeData dan ThemeMode.
- Menggunakan Theme.of(context) dan colorScheme agar tampilan mengikuti tema aktif.
- Menerapkan named routes, mengirim argumen ke halaman detail, dan mengatur rute dengan onGenerateRoute.
- Membuat navigasi tab bawah menggunakan NavigationBar dan IndexedStack.
- Menerapkan AnimatedContainer, AnimatedSwitcher, Hero, AnimationController, dan AnimatedOpacity.
- Menyimpan preferensi tema menggunakan SharedPreferences agar tetap tersedia setelah aplikasi dibuka kembali.
- Menambahkan Drawer dan halaman fallback 404 sebagai pengembangan navigasi.

3. Dasar Teori

3.1 ThemeData dan ThemeMode

ThemeData menyimpan konfigurasi tampilan seperti palet warna, tipografi, dan gaya komponen. ThemeMode menentukan apakah aplikasi menggunakan tema terang atau gelap. Pada kode latihan, seed color membentuk palet Material 3 dan ListenableBuilder membangun ulang MaterialApp ketika themeMode atau seedColor berubah.

```
ThemeData buatTema(Color seed, Brightness brightness) {
  return ThemeData(
    useMaterial3: true,
    colorSchemeSeed: seed,
    brightness: brightness,
    appBarTheme: const AppBarTheme(centerTitle: true),
  );
}
```

3.2 SharedPreferences

SharedPreferences merupakan penyimpanan key-value untuk preferensi sederhana. Pada aplikasi ini, nilai mode gelap disimpan sebagai bool dan warna tema disimpan sebagai integer dari nilai Color. Data dibaca sebelum runApp dijalankan.

3.3 Named Routes dan Argumen

Named routes memberi nama pada tujuan navigasi. Navigator.pushNamed membuka halaman berdasarkan nama rute dan dapat membawa objek melalui parameter arguments. onGenerateRoute digunakan untuk membangun halaman detail berdasarkan argumen yang diterima.

3.4 Hero dan Transisi Halaman

Hero menganimasikan elemen yang memiliki tag sama pada halaman asal dan tujuan. PageRouteBuilder memungkinkan pengembang menentukan transisi sendiri; pada latihan mandiri ketiga, FadeTransition digunakan untuk menampilkan halaman detail secara bertahap.

3.5 Animasi Implisit dan Eksplisit

Jenis	Contoh	Fungsi
Implisit	AnimatedContainer	Menganimasikan perubahan ukuran, warna, dan bentuk ketika nilainya berubah.
Implisit	AnimatedSwitcher	Menganimasikan pergantian widget anak, dengan ValueKey untuk membedakan nilai.
Implisit	AnimatedOpacity	Mengatur efek memudar atau muncul melalui nilai opacity.
Eksplisit	AnimationController + RotationTransition	Memberi kendali untuk memulai, mengulang, dan menghentikan animasi.

3.6 NavigationBar, IndexedStack, dan Drawer

NavigationBar menyediakan navigasi tab bawah. IndexedStack menampilkan halaman pada indeks aktif sekaligus mempertahankan state halaman lain. Drawer menyediakan menu navigasi tambahan yang dapat dibuka dari AppBar.

4. Alat dan Bahan

- Flutter SDK dan Dart SDK.
- Visual Studio Code atau Android Studio.
- Android Emulator atau perangkat Android.
- Proyek Flutter praktikum_6.
- Package shared_preferences.

Perintah persiapan proyek:

```
flutter create praktikum_6
cd praktikum_6
flutter pub add shared_preferences
flutter run
```

5. Implementasi Praktikum

5.1 Bagian A: Tema Terang/Gelap dan Warna Tema

Pada lib/main.dart, siapkan dua ValueNotifier global sesuai modul: themeMode untuk mode terang/gelap dan seedColor untuk warna dasar tema. Daftar pilihan warna berisi indigo, teal, deepOrange, dan pink.

```
final themeMode = ValueNotifier<ThemeMode>(ThemeMode.light);
final seedColor = ValueNotifier<Color>(Colors.indigo);
const pilihanWarna = [Colors.indigo, Colors.teal, Colors.deepOrange, Colors.pink];
```

Buat fungsi buatTema agar ThemeData menggunakan Material 3, colorSchemeSeed, dan brightness. AppBar juga diatur agar judul berada di tengah.

```
ThemeData buatTema(Color seed, Brightness brightness) {
  return ThemeData(
    useMaterial3: true,
    colorSchemeSeed: seed,
    brightness: brightness,
    appBarTheme: const AppBarTheme(centerTitle: true),
  );
}
```

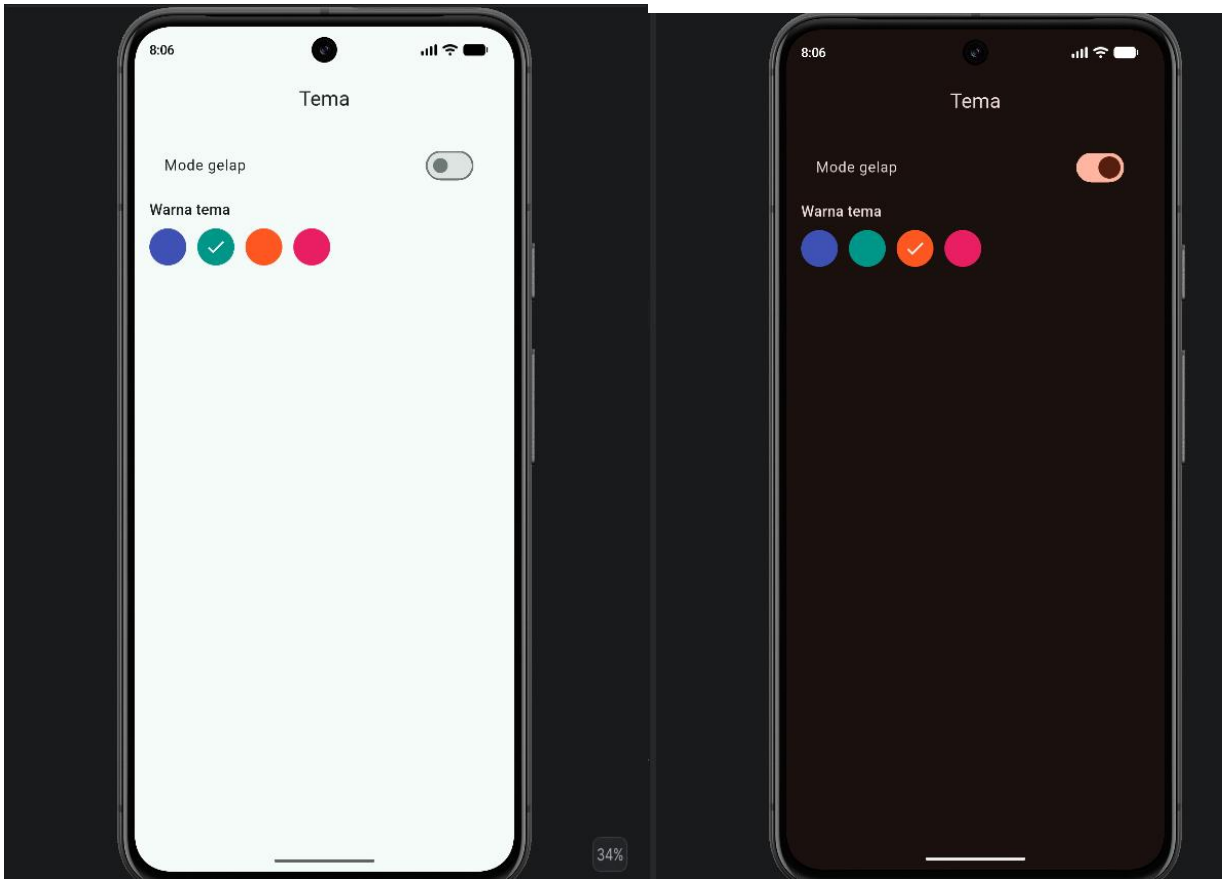
Di MyApp, bungkus MaterialApp dengan ListenableBuilder yang mendengarkan perubahan themeMode dan seedColor. Isi properti theme, darkTheme, dan themeMode dengan nilai dari notifier. Halaman awal pada bagian ini menampilkan PengaturanTab.

```
listenable: Listenable.merge([themeMode, seedColor]),
theme: buatTema(seedColor.value, Brightness.light),
darkTheme: buatTema(seedColor.value, Brightness.dark),
themeMode: themeMode.value,
```

Pada PengaturanTab, gunakan SwitchListTile untuk mengganti mode. Tampilkan pilihan warna dengan Wrap dan CircleAvatar; ketika warna disentuh, ubah seedColor.value. Gunakan Theme.of(context) untuk gaya teks, bukan menuliskan warna teks secara manual.

```
onChanged: (v) {
  themeMode.value = v ? ThemeMode.dark : ThemeMode.light;
},
onTap: () => seedColor.value = c,
```

Hasil:



Mode gelap dan pilihan warna langsung mengubah AppBar serta tampilan aplikasi tanpa perlu memulai ulang aplikasi.

5.2 Bagian B: Named Routes, Argumen, dan Hero

Tambahkan model Item dengan properti nama, ikon, dan warna. Siapkan daftarItem yang berisi Flutter, Musik, Kamera, dan Peta seperti contoh pada modul.

```
class Item {
  final String nama;
  final IconData ikon;
  final Color warna;
  const Item(this.nama, this.ikon, this.warna);
}
```

Buat BerandaTab menggunakan ListView.builder untuk menampilkan daftar item. Bungkus ikon di dalam Hero dengan tag unik berdasarkan nama item. Ketika ListTile ditekan, buka rute /detail dan kirim objek item melalui arguments.

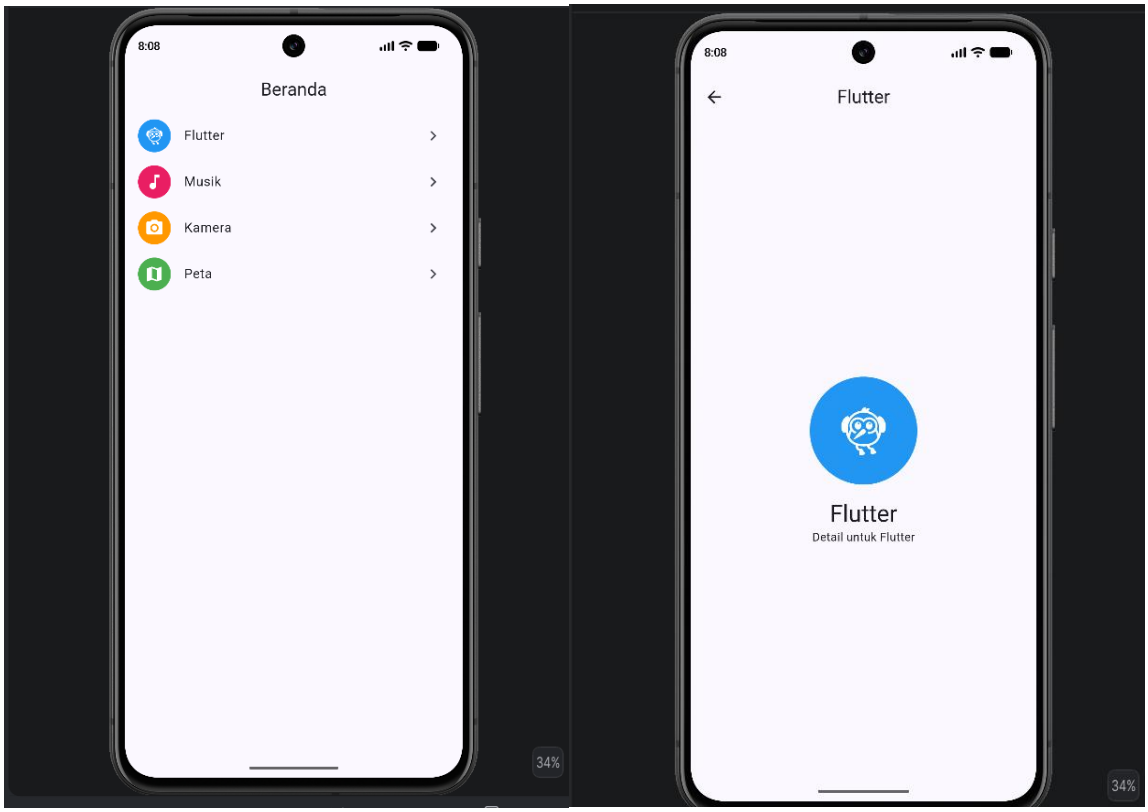
```
Navigator.pushNamed(context, '/detail', arguments: item);
```

Buat DetailPage yang menerima Item melalui konstruktor. Tampilkan ikon dan nama item, lalu gunakan Hero dengan tag yang sama seperti di BerandaTab agar perpindahan elemen dapat dianimasikan.

```
Hero(tag: 'ikon-${item.nama}', child: CircleAvatar(...))
```

Pada MyApp, atur initialRoute menjadi '/' dan daftarkan rute beranda pada routes. Tangani '/detail' melalui onGenerateRoute; ambil settings.arguments sebagai Item lalu buat MaterialPageRoute menuju DetailPage.

```
initialRoute: '/',
routes: {'/': (_) => Scaffold(body: const BerandaTab())},
onGenerateRoute: (settings) {
  if (settings.name == '/detail') {
    final item = settings.arguments as Item;
    return MaterialPageRoute(
      builder: (_) => DetailPage(item: item),
      settings: settings,
    );
  }
  return null;
},
```



Memilih item membuka halaman detail, ikon berpindah dan membesar melalui Hero, lalu kembali ke daftar dengan transisi sebaliknya

5.3 Bagian C: Animasi

Tambahkan AnimasiTab sebagai StatefulWidget. Pada state, gunakan SingleTickerProviderStateMixin karena demo ketiga memakai AnimationController. Siapkan _besar untuk AnimatedContainer dan _hitung untuk AnimatedSwitcher.

5.3.1 AnimatedContainer

AnimatedContainer menganimasikan perubahan ukuran, warna, dan radius sudut selama 500 milidetik. Tombol Ubah bentuk membalik nilai _besar melalui setState; nilai tersebut menentukan lebar, warna, dan radius.

```
AnimatedContainer(
  duration: const Duration(milliseconds: 500),
```

```
  curve: Curves.easeInOut,
  width: _besar ? 200 : 100,
  height: 100,
  decoration: BoxDecoration(
    color: _besar ? warna.primary : warna.tertiary,
    borderRadius: BorderRadius.circular(_besar ? 50 : 8),
  ),
)
```

5.3.2 AnimatedSwitcher

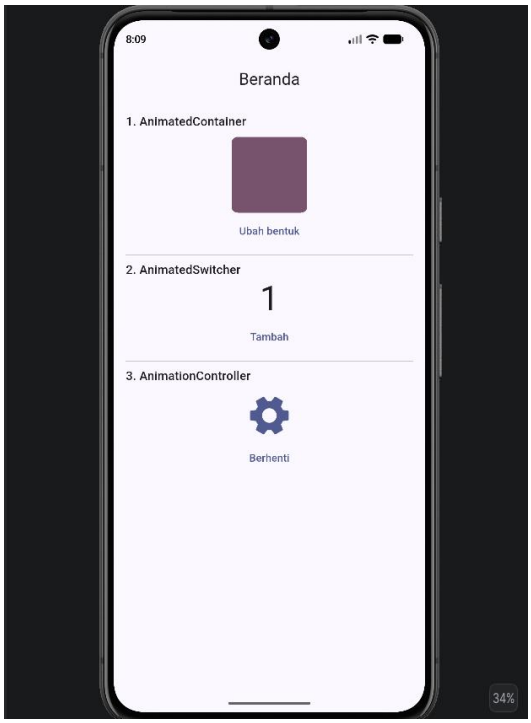
AnimatedSwitcher menampilkan perubahan angka penghitung dengan efek ScaleTransition. ValueKey(_hitung) membedakan widget teks setiap kali angka berubah, sehingga Flutter dapat menjalankan animasi pergantian.

```
AnimatedSwitcher(
  duration: const Duration(milliseconds: 300),
  transitionBuilder: (child, anim) => ScaleTransition(scale: anim, child: child),
  child: Text('$_hitung', key: ValueKey(_hitung)),
)
```

5.3.3 AnimationController dan RotationTransition

Buat AnimationController berdurasi dua detik di initState dan panggil repeat() agar ikon pengaturan terus berputar. Tombol Putar/Berhenti mengubah controller antara repeat() dan stop(). Lepaskan controller di dispose() ketika state tidak lagi digunakan.

```
_putar = AnimationController(
  vsync: this,
  duration: const Duration(seconds: 2),
)..repeat();
RotationTransition(turns: _putar, child: Icon(Icons.settings, size: 64))
@override
void dispose() {
  _putar.dispose();
  super.dispose();
}
```



Ubah bentuk mengubah bentuk secara halus, tombol Tambah mengganti angka dengan efek membesar, dan ikon roda gigi berputar serta dapat dihentikan

5.4 Bagian D: Navigasi Tab Bawah

Tambahkan ShellPage sebagai StatefulWidget dengan indeks tab aktif _index. Siapkan tiga halaman, yaitu BerandaTab, AnimasiTab, dan PengaturanTab, serta daftar judul yang sesuai.

```
static const _halaman = [BerandaTab(), AnimasiTab(), PengaturanTab()];
static const _judul = ['Beranda', 'Animasi', 'Pengaturan'];
```

Pada ShellPage, tampilkan judul AppBar berdasarkan _index. Gunakan IndexedStack untuk menampilkan halaman yang dipilih sambil mempertahankan state halaman lain.

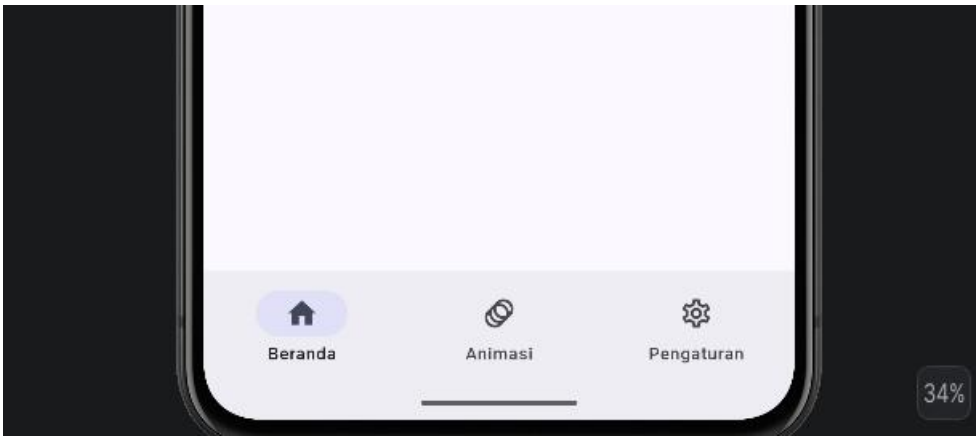
```
body: IndexedStack(index: _index, children: _halaman),
```

Tambahkan NavigationBar di bagian bawah. selectedIndex mengikuti _index, sedangkan onDestinationSelected memperbarui indeks melalui setState. Tambahkan tiga NavigationDestination untuk Beranda, Animasi, dan Pengaturan.

```
bottomNavigationBar: NavigationBar(
  selectedIndex: _index,
  onDestinationSelected: (i) => setState(() => _index = i),
  destinations: const [
    NavigationDestination(icon: Icon(Icons.home_outlined), selectedIcon:
Icon(Icons.home), label: 'Beranda'),
    NavigationDestination(icon: Icon(Icons.animation), label: 'Animasi'),
    NavigationDestination(icon: Icon(Icons.settings_outlined), selectedIcon:
Icon(Icons.settings), label: 'Pengaturan'),
  ],
),
```

Terakhir, ubah rute '/' pada MyApp agar membangun const ShellPage().

```
routes: {'/': (_) => const ShellPage()},
```



Aplikasi memiliki tiga tab bawah. Beranda membuka detail dengan Hero, Animasi menjalankan tiga demo, dan Pengaturan mengubah tema aplikasi termasuk pada halaman detail.

6. Latihan Mandiri

6.1 Latihan 1: Penyimpanan Tema

Berkas latihan1.dart menambahkan penyimpanan mode gelap dan warna tema menggunakan shared_preferences. Fungsi simpanModeGelap menyimpan nilai boolean pada key mode_gelap, sedangkan simpanWarnaTema menyimpan warna sebagai integer pada key warna_tema. Fungsi muatPengaturanTema membaca kedua nilai dan menggunakan nilai default apabila belum tersedia.

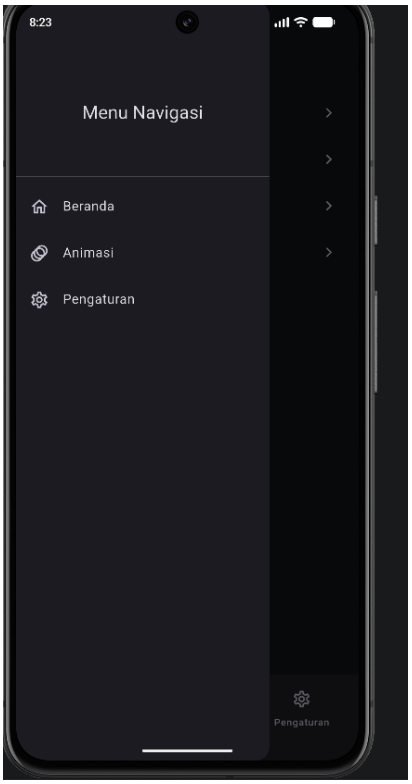
```
Future<void> simpanModeGelap(bool gelap) async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setBool('mode_gelap', gelap);
}

Future<void> simpanWarnaTema(Color warna) async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setInt('warna_tema', warna.value);
}
```

Pada main(), WidgetsFlutterBinding.ensureInitialized() dipanggil sebelum pemuatan preferensi asynchronous. runApp baru dijalankan setelah pengaturan dimuat, sehingga tema awal sesuai dengan pilihan terakhir pengguna.

6.2 Latihan 2: Drawer

Berkas latihan2.dart menambahkan Drawer pada ShellPage. Drawer berisi menu Beranda, Animasi, dan Pengaturan. Setiap menu mengubah _index lalu memanggil Navigator.pop(context) untuk menutup Drawer. Navigasi ini menjadi alternatif selain NavigationBar di bagian bawah layar.



6.3 Latihan 3: PageRouteBuilder dan FadeTransition

Berkas latihan3.dart mengganti MaterialPageRoute untuk halaman detail dengan PageRouteBuilder. pageBuilder membangun DetailPage, sedangkan transitionsBuilder membungkus halaman dengan FadeTransition. Nilai opacity mengikuti animation yang diberikan Flutter selama transisi rute.

```
return FadeTransition(
  opacity: animation,
  child: child,
);
```

6.4 Latihan 4: Penanganan Rute Tidak Ditemukan

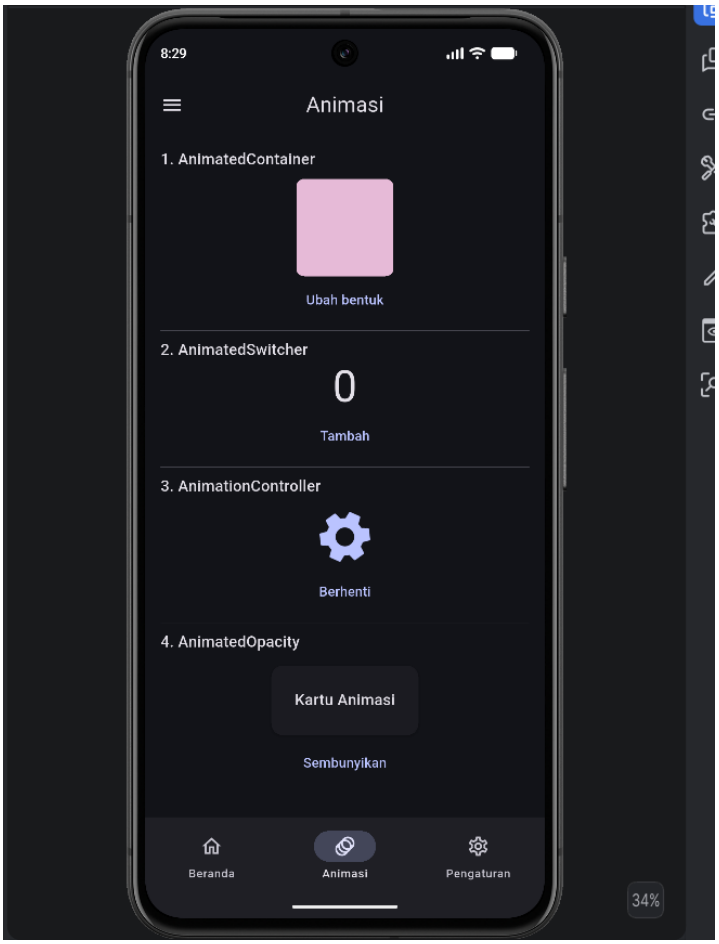
Berkas latihan4.dart menambahkan onUnknownRoute pada MaterialApp. Jika nama rute tidak dikenal dan tidak ditangani oleh routes maupun onGenerateRoute, aplikasi menampilkan halaman dengan pesan "404 - Halaman tidak ditemukan". Mekanisme ini memberi respons yang lebih jelas dibandingkan kegagalan navigasi tanpa halaman fallback.

6.5 Latihan 5: AnimatedOpacity

Berkas latihan5.dart menambahkan variabel _terlihat dan demo AnimatedOpacity sebagai animasi keempat. Tombol Sembunyikan/Tampilkan mengubah _terlihat melalui setState. Nilai opacity berubah antara 1.0 dan 0.0 selama 500 milidetik, sehingga kartu dapat dimunculkan atau disamarkan secara bertahap.

```
AnimatedOpacity(
  opacity: _terlihat ? 1.0 : 0.0,
  duration: const Duration(milliseconds: 500),
  child: Card(
```

```
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Text('Kartu Animasi'),
      ),
    ),
  )
```



7. Tugas: Aplikasi Galeri/Katalog Interaktif

7.1 Deskripsi

Tugas praktikum mengembangkan aplikasi bertema bebas, misalnya profil diri, katalog produk, atau galeri tempat wisata. Aplikasi menggabungkan navigasi tab, halaman detail, transisi, animasi, serta preferensi tema yang tersimpan.

- Minimal tiga tab menggunakan NavigationBar.
- Named routes dengan argumen untuk membuka halaman detail.
- Hero pada halaman daftar dan halaman detail dengan tag yang cocok.
- Minimal satu animasi implisit dan satu animasi eksplisit menggunakan AnimationController.
- Tema terang/gelap dan minimal tiga pilihan warna tema.
- Preferensi tema tetap tersedia setelah aplikasi ditutup dan dibuka kembali.

- Warna dan gaya teks mengambil nilai dari Theme.of(context) atau colorScheme, bukan hard-coded tanpa kebutuhan.

7.3 Catatan Implementasi

Bagian tugas berisi kkode aplikasi Galeri Wisata. Kode tersebut menggabungkan penyimpanan tema, katalog tempat wisata, navigasi tab, halaman detail dengan Hero, serta animasi implisit dan eksplisit.

7.4 Pengelolaan Tema dan Penyimpanan Preferensi

Aplikasi menggunakan dua ValueNotifier global, yaitu themeMode untuk menyimpan mode terang/gelap dan seedColor untuk menyimpan warna dasar tema. Daftar pilihanWarna berisi indigo, teal, deepOrange, dan pink. Fungsi buatTema membangun ThemeData Material 3 berdasarkan warna dasar dan tingkat kecerahan.

```
final themeMode = ValueNotifier<ThemeMode>(ThemeMode.light);
final seedColor = ValueNotifier<Color>(Colors.indigo);
```

Fungsi muatTema membaca preferensi dari SharedPreferences ketika aplikasi mulai dijalankan. Nilai gelap dibaca sebagai boolean dengan nilai awal false, sedangkan indeks warna dibaca dengan nilai awal 0. Fungsi simpanTema memperbarui ValueNotifier agar tampilan langsung berubah, lalu menyimpan pilihan mode dan indeks warna ke penyimpanan lokal.

```
final prefs = await SharedPreferences.getInstance();
final gelap = prefs.getBool('gelap') ?? false;
final idx = prefs.getInt('warna') ?? 0;
```

Pada main(), WidgetsFlutterBinding.ensureInitialized() dipanggil sebelum muatTema() agar pemanggilan plugin dapat dilakukan sebelum runApp. Aplikasi baru dijalankan setelah preferensi tema selesai dibaca. MyApp memakai ListenableBuilder untuk mendengarkan perubahan themeMode dan seedColor, kemudian membangun MaterialApp dengan theme, darkTheme, dan themeMode yang sesuai.

7.5 Model Data dan Daftar Wisata

Kelas Item menjadi model data untuk setiap tujuan wisata. Setiap objek menyimpan nama, lokasi, deskripsi, dan ikon. Konstanta daftarItem memuat empat contoh tempat wisata: Candi Borobudur di Magelang, Raja Ampat di Papua Barat, Gunung Bromo di Jawa Timur, dan Danau Toba di Sumatera Utara. Data ini ditampilkan pada tab Beranda dan dikirim ke halaman detail ketika dipilih.

```
class Item {
  final String nama;
  final String lokasi;
  final String deskripsi;
  final IconData ikon;
  const Item(this.nama, this.lokasi, this.deskripsi, this.ikon);
}
```

7.6 Navigasi Tab dengan NavigationBar dan IndexedStack

ShellPage menyimpan indeks tab aktif pada variabel _index. Daftar _halaman berisi BerandaTab, AnimasiTab, dan PengaturanTab, sedangkan _judul menyimpan judul AppBar untuk tiap tab. NavigationBar mengubah _index melalui setState saat pengguna memilih tujuan navigasi. IndexedStack

menampilkan halaman berdasarkan indeks tersebut dan mempertahankan state halaman lain ketika pengguna berpindah tab.

```
body: IndexedStack(index: _index, children: _halaman),
onDestinationSelected: (i) => setState(() => _index = i),
```

7.7 Halaman Beranda, Named Routes, Argumen, dan Hero

BerandaTab menggunakan ListView.builder untuk membuat daftar tempat wisata dari daftarItem. Setiap baris menampilkan ikon, nama, dan lokasi. Ikon dibungkus Hero dengan tag yang dibentuk dari nama item, misalnya ikon-Candi Borobudur. Saat baris dipilih, Navigator.pushNamed membuka rute /detail dan mengirim objek Item melalui arguments.

```
onTap: () => Navigator.pushNamed(context, '/detail', arguments: item),
```

Pada MaterialApp, rute / mengarah ke ShellPage. Rute /detail diproses oleh onGenerateRoute, yang membaca settings.arguments sebagai Item dan membuat DetailPage. DetailPage menampilkan ikon berukuran lebih besar, nama, lokasi, serta deskripsi wisata. Tag Hero pada halaman detail sama dengan tag di daftar, sehingga ikon dapat dianimasikan saat berpindah halaman.

```
if (settings.name == '/detail') {
  final item = settings.arguments as Item;
  return MaterialPageRoute(
    builder: (_) => DetailPage(item: item),
    settings: settings,
  );
}
```

7.8 Animasi pada Tab Animasi

AnimasiTab menyediakan dua demonstrasi. Pertama, AnimatedContainer mengubah lebar, warna, dan radius sudut berdasarkan nilai _besar. Tombol Ubah bentuk membalik nilai tersebut melalui setState, dan perubahan berlangsung selama 500 milidetik dengan kurva easeInOut. Ini merupakan animasi implisit karena Flutter menganimasikan perubahan properti secara otomatis.

```
AnimatedContainer(
  duration: const Duration(milliseconds: 500),
  curve: Curves.easeInOut,
  width: _besar ? 200 : 100,
  height: 100,
  decoration: BoxDecoration(
    color: _besar ? warna.primary : warna.tertiary,
    borderRadius: BorderRadius.circular(_besar ? 50 : 8),
  ),
)
```

Kedua, AnimationController dibuat di initState dengan durasi dua detik dan langsung dijalankan berulang melalui repeat(). RotationTransition menggunakan controller tersebut untuk memutar ikon explore. Tombol Putar/Berhenti memanggil stop() atau repeat() sesuai status animasi. Controller dilepas di dispose() agar sumber daya animasi tidak terus berjalan setelah widget dihapus.

```
_putar = AnimationController(
  vsync: this,
```

```
    duration: const Duration(seconds: 2),
  )..repeat();
  RotationTransition(
    turns: _putar,
    child: Icon(Icons.explore, size: 64, color: warna.primary),
  )
```

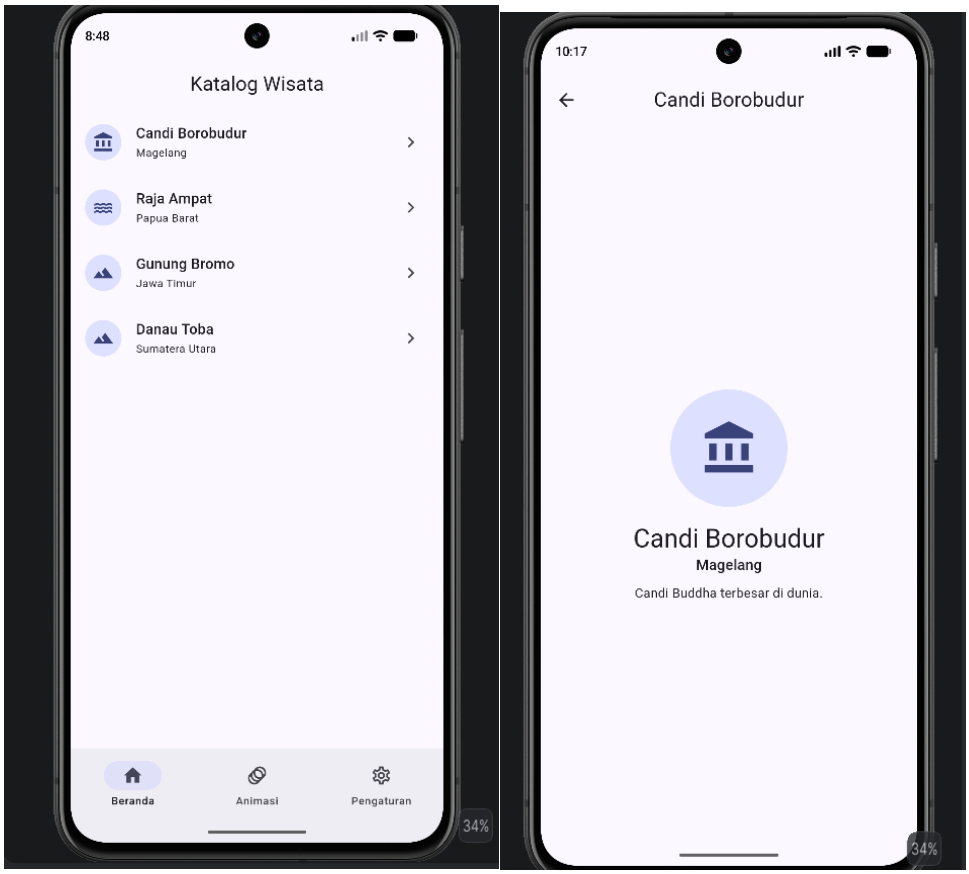
7.9 Pengaturan Tema

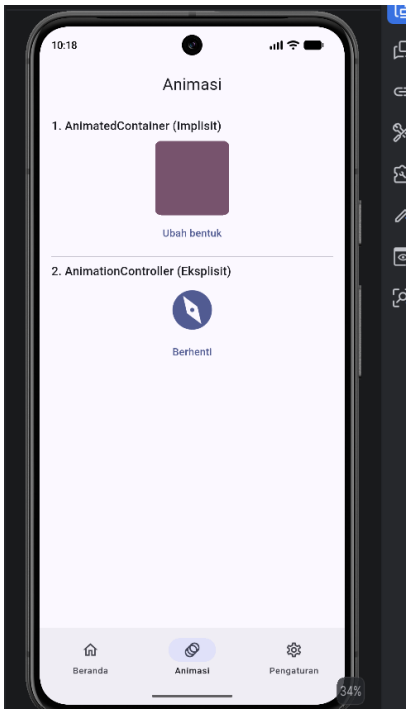
PengaturanTab menggunakan ListenableBuilder untuk memperbarui tampilan ketika mode atau warna tema berubah. SwitchListTile memanggil simpanTema saat mode gelap diaktifkan atau dinonaktifkan. Pilihan warna ditampilkan dalam bentuk CircleAvatar di dalam Wrap; warna yang sedang aktif diberi ikon centang. Pemilihan warna memanggil simpanTema dengan status mode saat ini dan warna yang dipilih.

```
onChanged: (v) => simpanTema(v, seedColor.value),
onTap: () => simpanTema(gelap, c),
```

7.10 Ringkasan Alur Aplikasi

Saat aplikasi dibuka, preferensi tema dimuat terlebih dahulu. Pengguna kemudian melihat tab Beranda yang berisi katalog empat tempat wisata. Memilih salah satu tempat membuka halaman detail melalui named route dan menampilkan transisi Hero. Tab Animasi berisi demo perubahan bentuk dan ikon berputar, sedangkan tab Pengaturan menyediakan saklar mode gelap dan empat pilihan warna. Pengaturan tema disimpan secara lokal agar dapat digunakan kembali ketika aplikasi dibuka berikutnya.





8. Pengujian

Tabel berikut merupakan skenario pengujian yang perlu dijalankan pada emulator atau perangkat. Kolom status sengaja disediakan sebagai "Belum diuji" agar dapat diubah berdasarkan hasil aktual setelah aplikasi dijalankan.

No	Skenario	Hasil yang Diharapkan	Status
1	Mengaktifkan mode gelap	Seluruh komponen bertema mengikuti mode gelap.	Berhasil
2	Memilih warna tema	Palet warna aplikasi berubah sesuai pilihan.	Berhasil
3	Menutup dan membuka aplikasi	Mode dan warna terakhir dimuat kembali.	Berhasil
4	Memilih item di Beranda	Halaman detail menampilkan item yang benar.	Berhasil
5	Membuka halaman detail	Ikon menggunakan transisi Hero dan halaman memudar masuk.	Berhasil
6	Menekan Ubah bentuk	Ukuran dan radius AnimatedContainer berubah halus.	Berhasil

7	Menekan Tambah	Angka bertambah dan AnimatedSwitcher berjalan.	Berhasil
8	Menekan Berhenti/Putar	Animasi rotasi berhenti atau berjalan kembali.	Berhasil
9	Menekan Sembunyikan/Tampilkan	Kartu memudar dan muncul kembali.	Berhasil
10	Berpindah tab lewat NavigationBar	Konten dan judul mengikuti tab aktif.	Berhasil
11	Berpindah tab lewat Drawer	Tab berubah dan Drawer tertutup.	Berhasil
12	Membuka rute tidak dikenal	Halaman fallback 404 ditampilkan.	Belum diuji

9. Hasil dan Pembahasan

Berdasarkan struktur kode yang tersedia, implementasi menggabungkan pengelolaan tema melalui ValueNotifier dan ListenableBuilder, penyimpanan preferensi dengan SharedPreferences, routing bernama, serta navigasi tiga tab. Halaman detail menerima objek Item melalui arguments dan memasangkan Hero pada ikon yang sama. Pada tab Animasi, tiga demonstrasi dasar menunjukkan animasi perubahan ukuran, pergantian angka, dan rotasi; latihan mandiri kelima menambahkan animasi opacity.

Latihan mandiri memperluas aplikasi secara bertahap. Penyimpanan tema membuat preferensi bertahan antar sesi, Drawer menyediakan cara navigasi kedua, PageRouteBuilder memberi kendali transisi rute, dan onUnknownRoute menyediakan fallback untuk nama rute yang tidak dikenali. Keberhasilan runtime dan kesesuaian visual tetap harus dikonfirmasi melalui pengujian langsung; karena itu status pengujian pada dokumen ini belum dinyatakan berhasil.

10. Kesimpulan

Praktikum Pertemuan 6 memperkenalkan penerapan tema, routing, navigasi, dan animasi pada Flutter dalam satu aplikasi. ThemeData dan ThemeMode digunakan untuk mengatur tampilan, SharedPreferences menyimpan preferensi tema, named routes dan argumen menghubungkan daftar dengan halaman detail, Hero dan PageRouteBuilder memberi transisi visual, sedangkan NavigationBar, IndexedStack, dan Drawer menyediakan pola navigasi. AnimatedContainer, AnimatedSwitcher, AnimationController, dan AnimatedOpacity menunjukkan pendekatan animasi implisit maupun eksplisit. Pengujian pada emulator tetap diperlukan untuk memverifikasi semua skenario.

11. Refleksi

11.1 Apa perbedaan animasi implisit dan eksplisit?

Animasi implisit seperti AnimatedContainer dan AnimatedOpacity cukup merespons perubahan nilai properti. Animasi eksplisit menggunakan AnimationController sehingga pengembang dapat mengendalikan mulai, berhenti, pengulangan, dan sinkronisasi animasi.

11.2 Mengapa tag Hero harus sama?

Tag yang sama membantu Flutter mengenali elemen yang mewakili objek visual yang sama pada dua halaman. Tag juga harus unik di antara Hero yang berada pada subtree halaman yang sama agar tidak terjadi konflik.

11.3 Apa keuntungan Theme.of(context)?

Theme.of(context) mengambil warna dan tipografi dari tema aktif sehingga tampilan dapat menyesuaikan mode terang/gelap dan pilihan seed color. Pendekatan ini mengurangi nilai warna manual yang tidak mengikuti tema.

11.4 Apa perbedaan IndexedStack dan Navigator?

IndexedStack memilih satu halaman dari beberapa widget yang tetap dipertahankan dalam struktur yang sama. Navigator mengelola perpindahan halaman melalui stack rute, misalnya membuka halaman detail di atas halaman sebelumnya.

11.5 Mengapa AnimationController harus di-dispose?

dispose() melepaskan sumber daya ticker/animasi saat State tidak lagi digunakan. Ini mencegah ticker berjalan setelah widget dihapus dan menghindari peringatan atau pemborosan sumber daya.

12. Referensi

- Modul Praktikum Flutter Fundamental, Pertemuan 6: Tema, Routing dan Navigasi, serta Animasi.
- Dokumentasi Flutter : Themes: https://docs.flutter.dev/cookbook/design/themes
- Dokumentasi Flutter : Named routes: https://docs.flutter.dev/cookbook/navigation/named-routes
- Dokumentasi Flutter : Animations: https://docs.flutter.dev/ui/animations
- Dokumentasi Flutter : Hero animations: https://docs.flutter.dev/ui/animations/hero-animations
- Package shared_preferences: https://pub.dev/packages/shared_preferences